IThub

Приватный конструктор. Перегрузка конструкторов, создание конструкторов из конструкторов. Статические элементы



Ключевое слово static

- используется когда необходимо определять член класса (поле или метод), который будет применяться независимо от любого объекта данного класса
- статические поля являются общими для всего класса, поэтому они могут использоваться **без создания** объектов класса
- могут напрямую вызывать только **другие статические** методы своего класса
- могут напрямую получать доступ только к **статическим переменным** своего класса
- **не могут ссылаться** на this или super
- если для инициализации статических переменных нужно выполнять вычисления, тогда можно объявить блок static, который выполняется в точности один раз, когда класс загружается впервые

IThub

IThub

Статические поля

- При создании объектов класса для каждого объекта создается своя копия нестатических обычных полей. А статические поля являются общими для всего класса. Поэтому они могут использоваться без создания объектов класса.

```java
public class Program {
    public static void main(String[] args) {
        Person tom = new Person();
        Person bob = new Person();
        tom.displayId(); // id = 1
        bob.displayId(); // id = 2
        System.out.println(Person.counter); // 3
        // изменяем Person.counter
        Person.counter = 8;
        Person sam = new Person();
        sam.displayId(); // id = 8
    }
}

class Person {
    private int id;
    static int counter = 1;

    Person() {
        id = counter++;
    }

    public void displayId() {
        System.out.printf("Id: %d \n", id);
    }
}
```

Класс Person содержит статическую переменную counter, которая увеличивается в конструкторе и ее значение присваивается переменной id. То есть при создании каждого нового объекта Person эта переменная будет увеличиваться, поэтому у каждого нового объекта Person значение поля id будет на 1 больше чем у предыдущего.

IThub

IThub

Статические методы

Статические методы также относятся ко всему классу в целом. Например, в примере раньше статическая переменная counter была доступна извне, и мы могли изменить ее значение вне класса Person. Сделаем ее недоступной для изменения извне, но доступной для чтения. Для этого используем статический метод:

```
public class Program {
    public static void main(String[] args) {
        Person.displayCounter(); // Counter: 1
        Person tom = new Person();
        Person bob = new Person();
        Person.displayCounter(); // Counter: 3
    }
}

class Person {
    private int id;
    private static int counter = 1;

    Person() {
        id = counter++;
    }

    // статический метод
    public static void displayCounter() {
        System.out.printf("Counter: %d \n", counter);
    }

    public void displayId() {
        System.out.printf("Id: %d \n", id);
    }
}
```

Вообще методы определяются как статические, когда методы не затрагивают состояние объекта, то есть его нестатические поля и константы, и для вызова метода нет смысла создавать экземпляр класса.

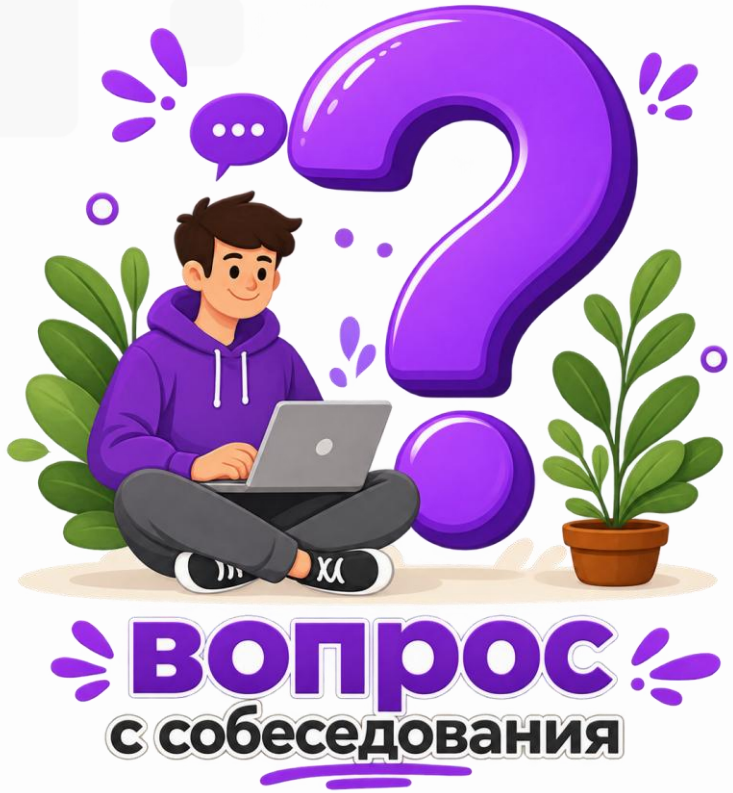
```
public class Program {
    public static void main(String[] args) {
        System.out.println(Operation.sum(45, 23));
        System.out.println(Operation.subtract(45, 23));
        System.out.println(Operation.multiply(4, 23));
    }
}

class Operation {
    static int sum(int x, int y) {
        return x + y;
    }

    static int subtract(int x, int y) {
        return x - y;
    }

    static int multiply(int x, int y) {
        return x * y;
    }
}
```

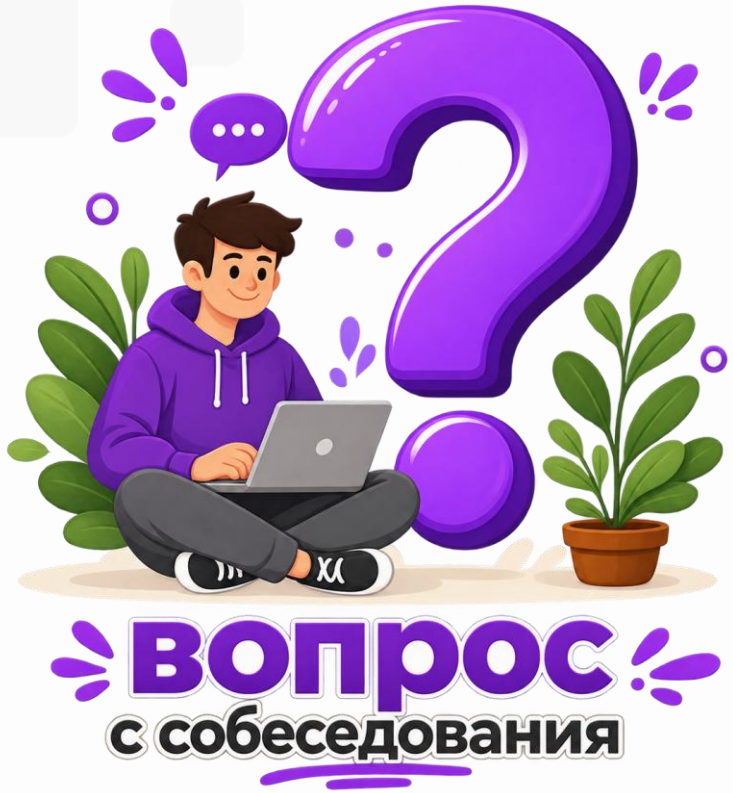
IThub



Может ли статический метод быть переопределён или перегружен?

Перегружен - да. Всё работает точно так же, как и с обычными методами - 2 статических метода могут иметь одинаковое имя, если количество их параметров или типов различается.

Переопределён - нет. Выбор вызываемого статического метода происходит при раннем связывании (на этапе компиляции, а не выполнения) и выполняться всегда будет родительский метод, хотя синтаксически переопределение статического метода - это вполне корректная языковая конструкция.



Могут ли нестатические методы перегрузить статические?

Да. В итоге получится два разных метода. Статический будет принадлежать классу и будет доступен через его имя, а нестатический будет принадлежать конкретному объекту и доступен через вызов метода этого объекта.

Вызов конструктора из конструктора

```
public class SomeClass {
    int a;
    int b;
    int c;

    public SomeClass(int a, int b) {
        this.a = a;
        this.b = b;
    }

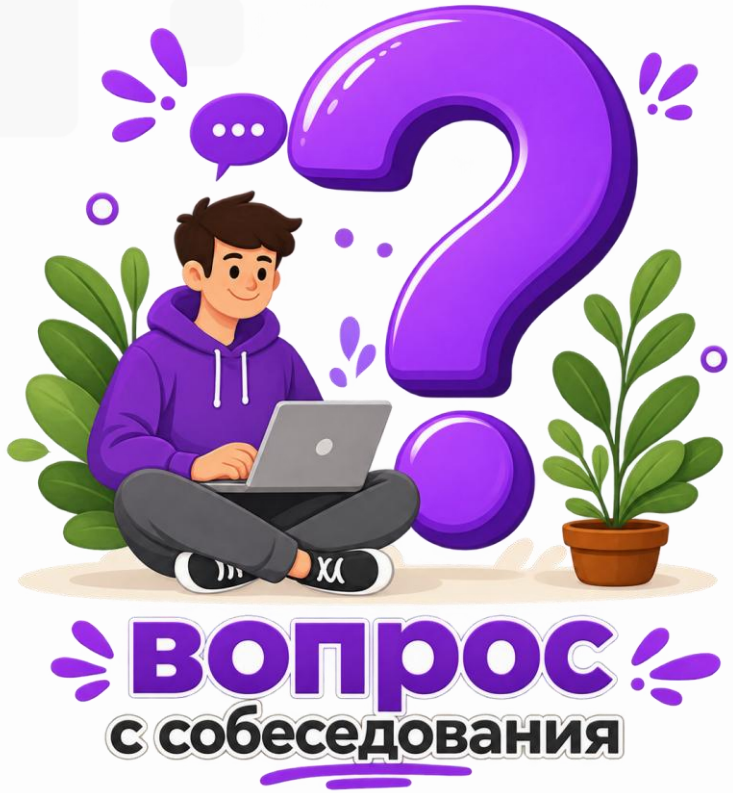
    public SomeClass(int a, int b, int c) {
        this(a, b);
        this.c = c;
    }

    public void show() {
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    }
}
```

Часто для того, чтобы избежать написание повторяющегося кода инициализации, необходимо вызвать код одного конструктора из другого.

Вызов другого конструктора (через this() или super()) должен быть либо первой строкой в конструкторе, либо его вообще не должно быть. Это жёсткое правило Java, и компилятор его не нарушит.

IThub



Где и как вы можете использовать приватный конструктор?

Приватный (помеченный ключевым словом private, скрытый) конструктор может использоваться публичным статическим методом генерации объектов данного класса. Также доступ к нему разрешён вложенным классам и может использоваться для их нужд.

IThub

Приватный конструктор

Приватные конструкторы позволяют нам ограничить создание экземпляров класса. То есть, они не дают создавать экземпляры класса вне самого класса.

Использование публичных и приватных конструкторов позволяет контролировать то, как мы будем создавать экземпляры класса - это называется делегированием конструктора.

```
public class PrivateConstructorClass {
    private PrivateConstructorClass() {
        // in the private constructor
    }
}
```

Делегирование конструктора

```
public class SomeClass {
   private final String value;
   private final String type;

   public SomeClass(int x) {
      this(Integer.toString(x), "int");
   }

   public SomeClass(boolean x) {
      this(Boolean.toString(x), "boolean");
   }

   private SomeClass(String value, String type) {
      this.value = value;
      this.type = type;
   }

   // getters and setters
}
```

SomeClass допускает инициализацию значением и типом, но мы хотим разрешить это только для подмножества типов. Общий конструктор должен быть закрытым, чтобы гарантировать использование только разрешенных типов.

Каждый из этих конструкторов вызывает общий частный конструктор для завершения инициализации объекта

Неинстанцируемые классы

Смысл неинстанцируемых классов в том, что мы не можем создать экземпляр такого класса. Создадим неинстанцируемый класс содержащий статические методы:

```
public class StringUtils {
    private StringUtils() {
    }

    public static String toUpperCase(String s) {
        return s.toUpperCase();
    }

    public static String toLowerCase(String s) {
        return s.toLowerCase();
    }
}
```

Мы можем использовать статические методы класса и без экземпляра этого класса

Задание (КТ №2) - менеджер задач с иерархией

Разработайте простое приложение для управления задачами с использованием понятий объектов, классов, инкапсуляции, полиморфизма и наследования на языке Java.

Создание классов:

Создайте базовый класс "Задача" (Task) с полями: название задачи, описание, дата создания. От этого базового класса унаследуйте класс "Задача с дедлайном" (DeadlineTask) с дополнительным полем: дата дедлайна. Ещё от базового класса унаследуйте класс "Повторяющаяся задача" (RecurringTask) с дополнительными полями: интервал повторения (например, "ежедневно", "еженедельно") и дата начала.

Менеджер задач:

Создайте класс "Менеджер задач" (TaskManager), который содержит список задач и позволяет добавлять, удалять и просматривать задачи. Реализуйте методы для добавления задач разных типов в список задач. Реализуйте методы для просмотра всех задач, сортированных по дате создания.

Инкапсуляция и геттеры/сеттеры:

Добавьте приватные поля в классы "Задача", "Задача с дедлайном" и "Повторяющаяся задача ". Реализуйте геттеры и сеттеры для всех полей, обеспечивая доступ к данным через методы.

Конструкторы:

В каждом классе реализуйте конструкторы для инициализации полей. Обязательно используйте конструкторы из базового класса при создании наследников.

Статические элементы:

В классе "Менеджер задач" реализуйте статическое поле, которое будет хранить количество созданных задач. Реализуйте статический метод для получения этого количества.

Тестирование:

В методе main создайте объекты разных типов задач, добавьте их в менеджер задач и выведите список задач. Продемонстрируйте использование геттеров и сеттеров для доступа к данным задач.